

TECHNICAL STANDARD OPERATING PROCEDURE

LAB-5: PostgreSQL 18.6 pgBackRest Full Backup & Restore on AWS EC2

DVD Rental Database | WAL Archiving | Full Backup | Repository Transfer | Restore to DB2

Item	Value
Source Server	DB1 - db1 - 172.30.2.177
Restore Server	DB2 - db2 - 172.30.2.128
Operating System	Amazon Linux 2023
PostgreSQL	18.6
pgBackRest	2.59.1
Stanza	lab5
Test Database	dvdrental
Backup Label	20260903-013127F
Repository	/var/lib/pgsql/pgbackrest_repo
Region	AWS us-east-2 (Ohio)
Lab Result	PASS - full backup and restore tested successfully
Tested By	Praveen Madupu
Lab Date	02 September 2026 - Wednesday

Index

#	Section	Page
1	Purpose and Final Outcome	3
2	Lab Architecture	3
3	Environment and Lab Inventory	3
4	pgBackRest Concepts Used	4
5	Phase 1 - Prepare DVD Rental Database on DB1	4
6	Phase 2 - Build and Install pgBackRest 2.59.1	5
7	Phase 3 - Configure Repository and Stanza	6
8	Phase 4 - Configure WAL Archiving and Validate	7
9	Phase 5 - Take Full pgBackRest Backup	7
10	Phase 6 - Capture DB1 Source Baseline	8
11	Phase 7 - Preserve DB2 Before Restore	8
12	Phase 8 - Install pgBackRest on DB2	8
13	Phase 9 - Transfer Repository through Windows PowerShell	9
14	Phase 10 - Configure and Validate Repository on DB2	10
15	Phase 11 - Restore Full Backup to DB2	10
16	Phase 12 - Pre-Startup Recovery Configuration Check	11
17	Phase 13 - Start DB2 and Validate Restore	12
18	Phase 14 - Validate DVD Rental Data and Write Test	12
19	Acceptance Checklist and Final Result	13
20	Post-Lab / Production Notes	14
21	Complete Working Command Reference	14
22	Official References and Document Control	16

1. Purpose and Final Outcome

LAB-5 demonstrates an end-to-end PostgreSQL 18.6 physical backup and restore using pgBackRest 2.59.1. A fresh DVD Rental sample database is protected on DB1, WAL is archived into the pgBackRest repository, a full backup is taken, the repository is transferred to DB2, and the backup is restored as an independent writable PostgreSQL cluster.

- Source: DB1 / 172.30.2.177 / PostgreSQL 18.6.
- Restore target: DB2 / 172.30.2.128 / PostgreSQL 18.6.
- Backup type: pgBackRest full backup.
- Backup label: 20260903-013127F.
- Source database validation: 15 public tables, key row counts matched, database size 15 MB.
- Restore validation: DB2 left recovery mode, became writable, matched the DVD Rental source baseline, and accepted a post-restore validation row.

2. Lab Architecture

```

DB1 - PostgreSQL 18.6
172.30.2.177
|
+-- dvdrental
|   +-- 15 public tables
|   +-- sample business data
|
v
pgBackRest 2.59.1
|
+-- WAL archive-push
+-- Full Backup: 20260903-013127F
|
v
/var/lib/pgsql/pgbackrest_repo
|
| package + transfer through Windows workstation
v
DB2 - PostgreSQL 18.6
172.30.2.128
|
+-- pgBackRest repository copy
+-- Full restore into /var/lib/pgsql/data
+-- WAL archive-get during recovery
v
dvdrental validated + writable
v
LAB-5 PASS

```

3. Environment and Lab Inventory

Component	DB1	DB2
Hostname	db1	db2
Private IPv4	172.30.2.177	172.30.2.128
Public IPv4	3.15.171.222	18.218.173.106
OS	Amazon Linux 2023	Amazon Linux 2023
PostgreSQL	18.6	18.6
PGDATA	/var/lib/pgsql/data	/var/lib/pgsql/data
Service	postgresql.service	postgresql.service
LAB-5 role	Backup source	Restore target

Setting	LAB-5 Value
pgBackRest version	2.59.1
Stanza	lab5
Repository	/var/lib/pgsql/pgbackrest_repo
Retention	repo1-retention-full=2
process-max	2
start-fast	y
Database	dvdrental
Backup label	20260903-013127F

4. pgBackRest Concepts Used

Component	Implementation
Stanza	lab5 - associates the PostgreSQL cluster with repository metadata
Repository	Local filesystem repository at /var/lib/pgsql/pgbackrest_repo
archive-push	PostgreSQL archive_command sends completed WAL to pgBackRest
check	Validates stanza, database/repository communication, and WAL archiving
Full backup	Complete physical backup of the PostgreSQL cluster
Backup set	20260903-013127F
archive-get	Restore-time command generated by pgBackRest to retrieve required WAL
Restore	Physical cluster restore into DB2 PGDATA

5. Phase 1 - Prepare DVD Rental Database on DB1

A fresh DVD Rental database was loaded before the LAB-5 full backup so the restore could be validated with known application data.

5.1 Download the tested sample SQL file

```
cd /tmp
curl -fL \
  "https://raw.githubusercontent.com/neondatabase/postgres-sample-dbs/main/dvdrental.sql" \
  -o /tmp/dvdrental.sql
ls -lh /tmp/dvdrental.sql
file /tmp/dvdrental.sql
grep -c "CREATE TABLE" /tmp/dvdrental.sql
```

5.2 Create and load the database

```
sudo -u postgres createdb \
  --template=template0 \
  --encoding=UTF8 \
  dvdrental

sudo -u postgres psql \
  -v ON_ERROR_STOP=1 \
  -d dvdrental \
  -f /tmp/dvdrental.sql
```

5.3 Validate the sample database

```
sudo -u postgres psql -d dvdrental -c "
SELECT COUNT(*) AS table_count
FROM pg_tables
WHERE schemaname='public';
"

sudo -u postgres psql -d dvdrental -c "
SELECT 'actor' AS table_name, COUNT(*) AS row_count FROM actor
UNION ALL SELECT 'customer', COUNT(*) FROM customer
UNION ALL SELECT 'film', COUNT(*) FROM film
UNION ALL SELECT 'inventory', COUNT(*) FROM inventory
UNION ALL SELECT 'payment', COUNT(*) FROM payment
UNION ALL SELECT 'rental', COUNT(*) FROM rental;
"

sudo -u postgres psql -d postgres -c "
SELECT pg_size_pretty(pg_database_size('dvdrental')) AS dvdrental_size;
"
```

Validation	Observed
Public tables	15
actor	200
customer	599
film	1000
inventory	4581
payment	14596
rental	16044
Database size	15 MB

6. Phase 2 - Build and Install pgBackRest 2.59.1 on DB1

pgBackRest 2.59.1 was built from source on Amazon Linux 2023 and installed as /usr/bin/pgbackrest.

6.1 Install build prerequisites

```
sudo dnf install -y \
gcc make meson ninja-build pkgconf-pkg-config \
openssl-devel libxml2-devel xz-devel bzip2-devel \
libyaml-devel libssh2-devel lz4-devel libpq-devel
```

6.2 Download source release

```
sudo mkdir -p /build
sudo chown ec2-user:ec2-user /build
cd /build
curl -fL \
"https://github.com/pgbackrest/pgbackrest/releases/download/release%2F2.59.1/pgbackrest-2.59.1.tar.gz" \
-o pgbackrest-2.59.1.tar.gz
tar -xzf pgbackrest-2.59.1.tar.gz
cd /build/pgbackrest-2.59.1
```

6.3 Configure, compile, and install

```
meson setup build
ninja -C build
sudo install -m 755 build/src/pgbackrest /usr/bin/pgbackrest
pgbackrest version
which pgbackrest
command -v pgbackrest
```

Validated pgBackRest version: 2.59.1. Installed binary: /usr/bin/pgbackrest.

7. Phase 3 - Configure Repository and Stanza on DB1

7.1 Create required directories

```
sudo mkdir -p -m 770 /var/log/pgbackrest
sudo chown postgres:postgres /var/log/pgbackrest

sudo mkdir -p /etc/pgbackrest/conf.d
sudo touch /etc/pgbackrest/pgbackrest.conf
sudo chown postgres:postgres /etc/pgbackrest/pgbackrest.conf
sudo chmod 640 /etc/pgbackrest/pgbackrest.conf

sudo mkdir -p /var/lib/pgsql/pgbackrest_repo
sudo chown postgres:postgres /var/lib/pgsql/pgbackrest_repo
sudo chmod 750 /var/lib/pgsql/pgbackrest_repo
```

7.2 Configure pgBackRest

```
sudo tee /etc/pgbackrest/pgbackrest.conf > /dev/null <<'EOF'
[lab5]
pg1-path=/var/lib/pgsql/data

[global]
repo1-path=/var/lib/pgsql/pgbackrest_repo
repo1-retention-full=2
start-fast=y
process-max=2
log-level-console=info
log-level-file=detail
EOF

sudo chown postgres:postgres /etc/pgbackrest/pgbackrest.conf
sudo chmod 640 /etc/pgbackrest/pgbackrest.conf
```

7.3 Create stanza

```
sudo -u postgres pgbackrest \
--stanza=lab5 \
stanza-create

sudo -u postgres pgbackrest \
--stanza=lab5 \
info
```

The stanza was created successfully. Before the first backup, a stanza with no valid backup is an expected initial state.

8. Phase 4 - Configure WAL Archiving and Validate

8.1 Switch PostgreSQL archive_command to pgBackRest

```
sudo -u postgres psql -d postgres -c \
'ALTER SYSTEM SET archive_command = 'pgbackrest --stanza=lab5 archive-push %p';'

sudo -u postgres psql -d postgres -c "SELECT pg_reload_conf();"

sudo -u postgres psql -d postgres -c "SHOW archive_mode;"
sudo -u postgres psql -d postgres -c "SHOW wal_level;"
sudo -u postgres psql -d postgres -c "SHOW archive_command;"
```

8.2 Force WAL and validate archiving

```
sudo -u postgres psql -d postgres -c "SELECT pg_switch_wal();"

sudo -u postgres psql -d postgres -c "
SELECT
  archived_count,
  last_archived_wal,
  last_archived_time,
  failed_count,
  last_failed_wal,
  last_failed_time
FROM pg_stat_archiver;
"

sudo -u postgres pgbackrest \
--stanza=lab5 \
check
```

Validated result: pgBackRest successfully archived WAL into repo1 and the pgBackRest check command completed successfully.

9. Phase 5 - Take Full pgBackRest Backup

```
sudo -u postgres pgbackrest \
--stanza=lab5 \
--type=full \
backup

sudo -u postgres pgbackrest \
--stanza=lab5 \
info

sudo -u postgres pgbackrest \
--stanza=lab5 \
--output=json \
info
```

Backup Attribute	Validated Value
Type	Full
Label	20260903-013127F
Start / Stop	2026-09-03 01:31:27+00 / 01:31:32+00
WAL start / stop	00000003000000000000000010 / same segment
LSN start / stop	0/10000028 / 0/10000158
Database backup size	45.9 MB
Repository backup size	6.7 MB
File total	1662
Stanza status	ok

10. Phase 6 - Capture DB1 Source Baseline

```
sudo -u postgres psql -d dvdrental -c "
SELECT COUNT(*) AS table_count
FROM pg_tables
WHERE schemaname='public';
"

sudo -u postgres psql -d dvdrental -c "
SELECT 'actor' AS table_name, COUNT(*) AS row_count FROM actor
UNION ALL SELECT 'customer', COUNT(*) FROM customer
UNION ALL SELECT 'film', COUNT(*) FROM film
UNION ALL SELECT 'inventory', COUNT(*) FROM inventory
UNION ALL SELECT 'payment', COUNT(*) FROM payment
UNION ALL SELECT 'rental', COUNT(*) FROM rental;
"

sudo -u postgres psql -d postgres -c "
SELECT pg_size_pretty(pg_database_size('dvdrental')) AS dvdrental_size;
"
```

This baseline was later reproduced on DB2 after restore: 15 public tables; actor 200; customer 599; film 1000; inventory 4581; payment 14596; rental 16044; database size 15 MB.

11. Phase 7 - Preserve DB2 Before Restore

DB2 contained the completed LAB-4 recovered cluster. It was stopped and preserved before LAB-5 reused /var/lib/pgsql/data.

```
hostname
sudo -u postgres psql -d postgres -c "
SELECT
  pg_is_in_recovery() AS in_recovery,
  current_database() AS database_name,
  version();
"

sudo systemctl stop postgresql
sudo systemctl is-active postgresql
ps -ef | grep '[p]ostgres'

sudo mv \
/var/lib/pgsql/data \
/var/lib/pgsql/data_before_lab5_pgbackrest_20260903

sudo ls -ld \
/var/lib/pgsql/data_before_lab5_pgbackrest_20260903
```

The original DB2 PGDATA was retained as a rollback/reference copy. The active PGDATA path was left free for the pgBackRest restore.

12. Phase 8 - Install pgBackRest on DB2

12.1 Install required runtime libraries

```
sudo dnf install -y libpq libssh2
rpm -q libpq libssh2
```

12.2 Transfer pgBackRest binary through Windows PowerShell

Run these commands from Windows PowerShell in C:\Users\DELL\Downloads.


```
scp -i "C:\Users\DELL\Downloads\pgsqlabssep2026.pem" `
ec2-user@3.15.171.222:/usr/bin/pgbackrest `
"C:\Users\DELL\Downloads\pgbackrest"

Get-Item "C:\Users\DELL\Downloads\pgbackrest"

scp -i "C:\Users\DELL\Downloads\pgsqlabssep2026.pem" `
"C:\Users\DELL\Downloads\pgbackrest" `
ec2-user@18.218.173.106:/tmp/pgbackrest
```

12.3 Install the binary on DB2

```
ls -lh /tmp/pgbackrest
sudo mv /tmp/pgbackrest /usr/bin/pgbackrest
sudo chown root:root /usr/bin/pgbackrest
sudo chmod 755 /usr/bin/pgbackrest
command -v pgbackrest
pgbackrest version
ldd /usr/bin/pgbackrest | grep "not found" || echo "PASS: all required shared libraries found"
```

Validated result: /usr/bin/pgbackrest, version 2.59.1, with all required shared libraries resolved.

13. Phase 9 - Transfer pgBackRest Repository through Windows PowerShell

13.1 Package repository on DB1

```
sudo tar -czf /tmp/lab5_pgbackrest_repo.tar.gz \
-C /var/lib/pgsql \
pgbackrest_repo

sudo chown ec2-user:ec2-user /tmp/lab5_pgbackrest_repo.tar.gz
ls -lh /tmp/lab5_pgbackrest_repo.tar.gz
```

13.2 Download repository package from DB1 to Windows

```
scp -i "C:\Users\DELL\Downloads\pgsqlabssep2026.pem" `
ec2-user@3.15.171.222:/tmp/lab5_pgbackrest_repo.tar.gz `
"C:\Users\DELL\Downloads\lab5_pgbackrest_repo.tar.gz"

Get-Item "C:\Users\DELL\Downloads\lab5_pgbackrest_repo.tar.gz"
```

13.3 Upload repository package from Windows to DB2

```
scp -i "C:\Users\DELL\Downloads\pgsqlabssep2026.pem" `
"C:\Users\DELL\Downloads\lab5_pgbackrest_repo.tar.gz" `
ec2-user@18.218.173.106:/tmp/lab5_pgbackrest_repo.tar.gz
```

Validated transferred repository package size on Windows: 9,374,885 bytes (about 9.0 MB / 9,155 KB as displayed by the transfer tools).

14. Phase 10 - Configure and Validate Repository on DB2

14.1 Create pgBackRest configuration

```
sudo mkdir -p /etc/pgbackrest/conf.d
sudo chown -R root:root /etc/pgbackrest
sudo chmod 755 /etc/pgbackrest
sudo chmod 755 /etc/pgbackrest/conf.d

sudo tee /etc/pgbackrest/pgbackrest.conf > /dev/null <<'EOF'
[lab5]
pg1-path=/var/lib/pgsql/data

[global]
repo1-path=/var/lib/pgsql/pgbackrest_repo
repo1-retention-full=2
start-fast=y
process-max=2
log-level-console=info
log-level-file=detail
EOF

sudo chown postgres:postgres /etc/pgbackrest/pgbackrest.conf
sudo chmod 640 /etc/pgbackrest/pgbackrest.conf
```

14.2 Extract and validate the repository

```
sudo tar -xzf /tmp/lab5_pgbackrest_repo.tar.gz \
-C /var/lib/pgsql

sudo chown -R postgres:postgres \
/var/lib/pgsql/pgbackrest_repo

cd /tmp
sudo -u postgres pgbackrest \
--stanza=lab5 \
info

sudo -u postgres pgbackrest \
--stanza=lab5 \
--output=json \
info
```

DB2 successfully read the copied repository and identified full backup set 20260903-013127F with stanza status ok.

15. Phase 11 - Restore Full Backup to DB2

15.1 Create empty restore target

```
sudo systemctl is-active postgresql
sudo test ! -e /var/lib/pgsql/data \
&& echo "PASS: PGDATA is free"

sudo mkdir -p /var/lib/pgsql/data
sudo chown postgres:postgres /var/lib/pgsql/data
sudo chmod 700 /var/lib/pgsql/data
sudo ls -ld /var/lib/pgsql/data
```

15.2 Restore the selected full backup

```
cd /tmp
sudo -u postgres pgbackrest \
--stanza=lab5 \
--set=20260903-013127F \
restore
```

Validated result: pgBackRest restored backup set 20260903-013127F successfully. Restore size was 45.9 MB with 1662 backup files, and pg_control was restored last.

15.3 Validate restored PGDATA

```
sudo -u postgres bash -c '
echo "PG_VERSION:"
cat /var/lib/pgsql/data/PG_VERSION
echo
echo "PGDATA file count:"
find /var/lib/pgsql/data -type f | wc -l
echo
echo "PGDATA ownership:"
ls -ld /var/lib/pgsql/data
'

sudo -u postgres /usr/bin/pg_controldata /var/lib/pgsql/data | \
grep -E "Database cluster state|Database system identifier|Latest checkpoint.s TimeLineID"
```

16. Phase 12 - Pre-Startup Recovery Configuration Check

pgBackRest created recovery.signal and appended archive-get as restore_command. Because the physical backup carried a historical primary_conninfo from the source cluster history, that replication connection setting was removed before starting this independent restore.

```
sudo cp \
/var/lib/pgsql/data/postgresql.auto.conf \
/var/lib/pgsql/data/postgresql.auto.conf.before_lab5_cleanup

sudo sed -i \
'/^[[:space:]]*primary_conninfo[[:space:]]*=/d' \
/var/lib/pgsql/data/postgresql.auto.conf

sudo chown postgres:postgres \
/var/lib/pgsql/data/postgresql.auto.conf \
/var/lib/pgsql/data/postgresql.auto.conf.before_lab5_cleanup

sudo cat /var/lib/pgsql/data/postgresql.auto.conf

sudo test -f /var/lib/pgsql/data/recovery.signal \
&& echo "PASS: recovery.signal exists"

sudo test ! -f /var/lib/pgsql/data/standby.signal \
&& echo "PASS: standby.signal does not exist"

sudo grep -n "^primary_conninfo" \
/var/lib/pgsql/data/postgresql.auto.conf \
|| echo "PASS: stale primary_conninfo removed"
```

Effective restore settings retained before startup: wal_level=replica, archive_mode=on, archive_command using pgBackRest archive-push, and restore_command using pgBackRest archive-get.

17. Phase 13 - Start DB2 and Validate Restore

```
sudo systemctl start postgresql
sudo systemctl is-active postgresql

sudo -u postgres psql -d postgres -c "
SELECT
    pg_is_in_recovery() AS in_recovery,
    current_setting('transaction_read_only') AS transaction_read_only;
"

sudo -u postgres psql -d postgres -c "
SELECT
    timeline_id,
    redo_lsn,
    checkpoint_lsn
FROM pg_control_checkpoint();
"

sudo test ! -f /var/lib/pgsql/data/recovery.signal \
&& echo "PASS: recovery.signal removed after successful recovery"
```

Validation	Observed
PostgreSQL service	active
pg_is_in_recovery()	false
transaction_read_only	off
Timeline ID	4
Redo LSN	0/11000028
Checkpoint LSN	0/11000028
recovery.signal after recovery	removed

18. Phase 14 - Validate DVD Rental Data and Write Test

18.1 Compare restored database with DB1 baseline

```
sudo -u postgres psql -d dvdrental -c "
SELECT COUNT(*) AS table_count
FROM pg_tables
WHERE schemaname='public';
"

sudo -u postgres psql -d dvdrental -c "
SELECT 'actor' AS table_name, COUNT(*) AS row_count FROM actor
UNION ALL SELECT 'customer', COUNT(*) FROM customer
UNION ALL SELECT 'film', COUNT(*) FROM film
UNION ALL SELECT 'inventory', COUNT(*) FROM inventory
UNION ALL SELECT 'payment', COUNT(*) FROM payment
UNION ALL SELECT 'rental', COUNT(*) FROM rental;
"

sudo -u postgres psql -d postgres -c "
SELECT pg_size_pretty(pg_database_size('dvdrental')) AS dvdrental_size;
"
```

Object / Metric	DB1 Source	DB2 Restored
Public tables	15	15
actor	200	200
customer	599	599

film	1000	1000
inventory	4581	4581
payment	14596	14596
rental	16044	16044
Database size	15 MB	15 MB

18.2 Prove DB2 is writable after restore

```

sudo -u postgres psql -d dvdrental -c "
CREATE TABLE IF NOT EXISTS lab5_restore_validation
(
    id    bigserial PRIMARY KEY,
    test_note text NOT NULL,
    created_at timestamptz DEFAULT now()
);
"

sudo -u postgres psql -d dvdrental -c "
INSERT INTO lab5_restore_validation(test_note)
VALUES ('LAB-5 pgBackRest Full Restore Validation on DB2');
"

sudo -u postgres psql -d dvdrental -c "
SELECT
    id,
    test_note,
    created_at
FROM lab5_restore_validation
ORDER BY id;
"

```

Validated row: ID 1, "LAB-5 pgBackRest Full Restore Validation on DB2", created at 2026-09-03 02:15:58.229859+00.

19. Acceptance Checklist and Final Result

Validation	Status
DVD Rental database loaded on DB1	PASS
15 public tables validated on DB1	PASS
pgBackRest 2.59.1 installed on DB1	PASS
Repository and stanza lab5 created	PASS
archive-push configured and WAL archived	PASS
pgBackRest check completed	PASS
Full backup 20260903-013127F completed	PASS
DB1 source row-count baseline captured	PASS
Previous DB2 PGDATA preserved	PASS
pgBackRest 2.59.1 installed on DB2	PASS
Repository package transferred to DB2	PASS
DB2 pgBackRest info status ok	PASS
Full backup restored into DB2 PGDATA	PASS
Recovery settings validated before startup	PASS
DB2 recovery completed and service active	PASS

DB2 is writable and not in recovery	PASS
DB2 DVD Rental counts match DB1	PASS
Post-restore write validation completed	PASS
FINAL LAB-5 RESULT	PASS

Final result: LAB-5 completed successfully. The tested full pgBackRest backup on DB1 was restored to DB2, the DVD Rental database matched the source baseline, and the restored server accepted a new write.

20. Post-Lab / Production Notes

- The local filesystem repository used in this lab is suitable for learning and controlled testing. Production backup repositories should be isolated from the database host and protected against host-level failure.
- For production, prefer a durable remote repository design such as dedicated backup storage or object storage, with encryption, access controls, monitoring, retention, and recovery testing.
- A backup is only proven when restore testing succeeds. LAB-5 validates both backup creation and actual recovery on a second server.
- Monitor archive_command/pg_stat_archiver and pgBackRest check/info. WAL archiving failures can break recoverability even when an older full backup exists.
- Retain enough full/differential/incremental backups and WAL according to business RPO/RTO and retention requirements.
- The physical restore reproduces cluster-level configuration from the backup. Review environment-specific connection, replication, archive, port, path, and authentication settings before starting a restored server.
- Do not use the Windows workstation as a production backup transport pattern. It was used here as a controlled lab transfer path between two EC2 instances.

21. Complete Working Command Reference

The following condensed command sets provide a clean repeatable reference for the completed LAB-5.

21.1 DB1 - backup source

```
# DVD Rental
cd /tmp
curl -fL "https://raw.githubusercontent.com/neondatabase/postgres-sample-dbs/main/dvdrental.sql" -o /tmp/dvdrental.sql
sudo -u postgres createdb --template=template0 --encoding=UTF8 dvdrental
sudo -u postgres psql -v ON_ERROR_STOP=1 -d dvdrental -f /tmp/dvdrental.sql

# pgBackRest config
sudo mkdir -p -m 770 /var/log/pgbackrest
sudo chown postgres:postgres /var/log/pgbackrest
sudo mkdir -p /etc/pgbackrest/conf.d
sudo mkdir -p /var/lib/pgsql/pgbackrest_repo
sudo chown postgres:postgres /var/lib/pgsql/pgbackrest_repo
sudo chmod 750 /var/lib/pgsql/pgbackrest_repo

sudo tee /etc/pgbackrest/pgbackrest.conf > /dev/null <<'EOF'
[lab5]
pg1-path=/var/lib/pgsql/data
[global]
repo1-path=/var/lib/pgsql/pgbackrest_repo
repo1-retention-full=2
start-fast=y
process-max=2
log-level-console=info
log-level-file=detail
EOF
sudo chown postgres:postgres /etc/pgbackrest/pgbackrest.conf
sudo chmod 640 /etc/pgbackrest/pgbackrest.conf

sudo -u postgres pgbackrest --stanza=lab5 stanza-create
sudo -u postgres psql -d postgres -c "ALTER SYSTEM SET archive_command = 'pgbackrest --stanza=lab5 archive-push %p';"
sudo -u postgres psql -d postgres -c "SELECT pg_reload_conf();"
sudo -u postgres psql -d postgres -c "SELECT pg_switch_wal();"
sudo -u postgres pgbackrest --stanza=lab5 check
sudo -u postgres pgbackrest --stanza=lab5 --type=full backup
sudo -u postgres pgbackrest --stanza=lab5 info

# Package repository
sudo tar -czf /tmp/lab5_pgbackrest_repo.tar.gz -C /var/lib/pgsql pgbackrest_repo
sudo chown ec2-user:ec2-user /tmp/lab5_pgbackrest_repo.tar.gz
```

21.2 Windows PowerShell - transfer binary and repository

```
scp -i "C:\Users\DELL\Downloads\pgsqlabssep2026.pem" `
ec2-user@3.15.171.222:/usr/bin/pgbackrest `
"C:\Users\DELL\Downloads\pgbackrest"

scp -i "C:\Users\DELL\Downloads\pgsqlabssep2026.pem" `
"C:\Users\DELL\Downloads\pgbackrest" `
ec2-user@18.218.173.106:/tmp/pgbackrest

scp -i "C:\Users\DELL\Downloads\pgsqlabssep2026.pem" `
ec2-user@3.15.171.222:/tmp/lab5_pgbackrest_repo.tar.gz `
"C:\Users\DELL\Downloads\lab5_pgbackrest_repo.tar.gz"

scp -i "C:\Users\DELL\Downloads\pgsqlabssep2026.pem" `
"C:\Users\DELL\Downloads\lab5_pgbackrest_repo.tar.gz" `
ec2-user@18.218.173.106:/tmp/lab5_pgbackrest_repo.tar.gz
```

21.3 DB2 - restore target

```

sudo systemctl stop postgresql
sudo mv /var/lib/pgsql/data /var/lib/pgsql/data_before_lab5_pgbackrest_20260903

sudo dnf install -y libpq libssh2
sudo mv /tmp/pgbackrest /usr/bin/pgbackrest
sudo chown root:root /usr/bin/pgbackrest
sudo chmod 755 /usr/bin/pgbackrest
pgbackrest version

sudo mkdir -p /etc/pgbackrest/conf.d
sudo tee /etc/pgbackrest/pgbackrest.conf > /dev/null <<'EOF'
[lab5]
pg1-path=/var/lib/pgsql/data
[global]
repo1-path=/var/lib/pgsql/pgbackrest_repo
repo1-retention-full=2
start-fast=y
process-max=2
log-level-console=info
log-level-file=detail
EOF
sudo chown postgres:postgres /etc/pgbackrest/pgbackrest.conf
sudo chmod 640 /etc/pgbackrest/pgbackrest.conf

sudo tar -xzf /tmp/lab5_pgbackrest_repo.tar.gz -C /var/lib/pgsql
sudo chown -R postgres:postgres /var/lib/pgsql/pgbackrest_repo
cd /tmp
sudo -u postgres pgbackrest --stanza=lab5 info

sudo mkdir -p /var/lib/pgsql/data
sudo chown postgres:postgres /var/lib/pgsql/data
sudo chmod 700 /var/lib/pgsql/data
sudo -u postgres pgbackrest --stanza=lab5 --set=20260903-013127F restore

sudo cp /var/lib/pgsql/data/postgresql.auto.conf /var/lib/pgsql/data/postgresql.auto.conf.before_lab5_cleanup
sudo sed -i 's/^[:space:]]*primary_conninfo[:space:]]*=/d' /var/lib/pgsql/data/postgresql.auto.conf
sudo chown postgres:postgres /var/lib/pgsql/data/postgresql.auto.conf /var/lib/pgsql/data/postgresql.auto.conf.before_lab5_cleanup

sudo systemctl start postgresql
sudo systemctl is-active postgresql

```

22. Official References and Document Control

Official technical references for the pgBackRest portions of this lab:

- pgBackRest User Guide - RHEL / PostgreSQL installation and configuration guidance: <https://pgbackrest.org/user-guide-rhel.html>
- pgBackRest Command Reference - stanza-create, check, backup, info, restore, archive-push, and archive-get: <https://pgbackrest.org/command.html>
- pgBackRest Release Information - version 2.59.1: <https://pgbackrest.org/release.html>
- PostgreSQL 18 - Continuous Archiving and Point-in-Time Recovery: <https://www.postgresql.org/docs/18/continuous-archiving.html>
- PostgreSQL 18 - Server Configuration / WAL: <https://www.postgresql.org/docs/18/runtime-config-wal.html>

Lab evidence used for this SOP: completed DB1/DB2 command outputs and Windows PowerShell transfer logs supplied during the LAB-5 session. The layout follows the prior LAB-4 technical SOP template. [filecite2turn20file12](#)

Field	Value
Document	LAB-5 PostgreSQL 18.6 pgBackRest Full Backup & Restore on AWS EC2
Status	Final - Tested

PostgreSQL 18.6 | AWS EC2 | LAB-5 pgBackRest Full Backup & Restore

Result	PASS
Tested By	Praveen Madupu
Date	02 September 2026 - Wednesday
Backup Label	20260903-013127F
Scope Control	Clean successful lab procedure and working scripts